



O Problema da IA Centralizada

Desafios técnicos e econômicos que motivaram o desenvolvimento da arquitetura local-first.



ERR_DEPENDENCY

Dependência Excessiva

Acesso a LLMs restrito a APIs proprietárias (OpenAI, Anthropic), criando "vendor lock-in" e vulnerabilidade a instabilidades de serviço externo.



ERR_HIGH_COST

Custos Elevados

Modelos de cobrança por token tornam inviável a experimentação acadêmica em larga escala e o uso contínuo em aplicações intensivas.



ERR_PRIVACY

Privacidade e Controle

Envio obrigatório de dados sensíveis para servidores de terceiros (Black Box), impossibilitando o uso em contextos de sigilo ou dados médicos/pessoais.

A Proposta MLX-Pilot

Uma plataforma desktop Local-First projetada para devolver o controle, reduzir custos e habilitar a pesquisa avançada em IA.



01

Execução Local

Capacidade de rodar LLMs (Llama 3, Mistral, Gemma) diretamente no hardware do usuário, garantindo privacidade total dos dados e zero latência de rede.

STATUS: OFFLINE-READY



02

Multi-Backend

Arquitetura agnóstica que unifica **MLX**, **llama.cpp** e **Ollama** sob uma única interface e API, permitindo flexibilidade total de infraestrutura.

STATUS: UNIFIED_API



03

Experimentação

Sandbox controlado para estudantes e pesquisadores testarem hiperparâmetros, quantizações e arquiteturas de modelos sem custos de API.

STATUS: ACADEMIC_MODE



04

Laboratório Prático

Transforma qualquer computador com GPU/NPU em uma estação de trabalho de IA, servindo como base para disciplinas de Computação e TCCs.

STATUS: LAB_INSTALLED

SYSTEM ARCHITECTURE // LOCAL-FIRST

POWERED BY RUST + TAURI

Visão Arquitetural

CORE::RUST // UI::TAURI // PROVIDERS::MODULAR

Rust Core



Alta performance e memory safety

Daemon HTTP



Axum + Tokio (Async Runtime)

Sistema Modular



Arquitetura baseada em Providers

UI Desktop



Tauri (Webview nativa leve)

Backends Suportados

PROVIDER_SELECTION_MODE = "AUTO"

MLX

APPLE SILICON NATIVE



Framework de arrays otimizado para arquitetura Apple, permitindo acesso direto à memória unificada.

METAL API

UNIFIED MEMORY

✓ Otimização nativa para chips M1/M2/M3

✓ Lazy evaluation para alta performance

✓ Suporte a modelos Fine-tuned localmente

NPU UTILIZATION

HIGH

llama.cpp

CROSS-PLATFORM CORE



Motor de inferência C++ puro focado em máxima eficiência e compatibilidade de hardware.

GGUF FORMAT

CPU + GPU

✓ Quantização Avançada (4-bit, 8-bit)

✓ Suporte Híbrido (Offload parcial p/ GPU)

✓ Execução em hardware legado (AVX2)

COMPATIBILITY

MAX

Ollama

SERVICE INTEGRATION



Camada de serviço que simplifica o gerenciamento de modelos e exposição via API.

REST API

MODEL MANAGER

✓ Download e gestão automática de Blobs

✓ Interface unificada para múltiplos modelos

✓ Fácil integração com ferramentas externas

EASE OF USE

HIGH

BACKEND_STATUS: ACTIVE ● MLX ● LLAMA ● OLLAMA

PI_IV // ARCHITECTURE_V1

Diferenciais Técnicos

SYSTEM_CAPABILITIES::V1.0 // INNOVATION_MODULE



Arquitetura Modular

Sistema plugável onde cada componente (UI, Core, Providers) opera de forma desacoplada, facilitando manutenção e upgrades.

```
trait Provider { fn infer(&self, prompt: str); }
```



Multi-Provider

Camada de abstração unificada que permite alternar entre backends (MLX, Llama, Ollama) em tempo de execução sem reiniciar.

```
let backend = ProviderFactory::create(config.type);
```



Streaming Real-time

Implementação eficiente de Server-Sent Events (SSE) para entrega de tokens com latência mínima na interface do usuário.

```
response.stream().map(|chunk|  
Event::default().data(chunk))
```



Catálogo Hugging Face

Integração nativa com a API do Hugging Face para busca, filtragem e download automatizado de modelos GGUF e MLX diretamente pela interface.

```
GET https://huggingface.co/api/models?search=llama-3&filter=gguf
```

```
> searching models...  
> found: llama-3-8b-
```



Base para Agentes

Estrutura preparada para loop de feedback e tool-use, pavimentando o caminho para agentes autônomos locais.

```
agent.think().act().observe();
```

Demonstração Funcional

CURRENT_BUILD: v0.8.2-alpha

Chat com Streaming

REAL-TIME TOKEN GENERATION



Interface de chat responsiva utilizando Server-Sent Events (SSE) para exibir tokens assim que são inferidos, sem latência perceptível.

User: Explique Rust em uma frase.

Bot: Rust é uma linguagem de sistemas focada em segurança e concorrência.

TOK/S: 45.2

LATENCY: 12ms

Model Manager

HUGGING FACE INTEGRATION



Catálogo integrado para busca e download direto de modelos (GGUF/MLX) com verificação automática de hash e compatibilidade.

Llama-3-8B-Instruct-v0.2.Q4_K_M.gguf

3.8GB / 5.2GB

SOURCE: HF_HUB

DOWNLOADING...

Seleção de Backend

HOT-SWAPPABLE PROVIDERS



Capacidade de alternar dinamicamente entre motores de inferência (MLX, Llama.cpp, Ollama) sem reiniciar a aplicação.

Select Provider:

MLX (Apple)

llama.cpp

Ollama

> Provider switched successfully.

> Loaded 8B parameters in VRAM.

Execução Local

PRIVACY & OFFLINE FIRST



Ambiente totalmente isolado que garante que nenhum dado do chat ou telemetria deixe a máquina local do usuário.

Network Status

DISCONNECTED

Local Server: Port 3000

0 External Requests Blocked

Simulação de Desempenho de Hardware

Adição de peças de CPU, RAM, GPU, VRAM e outros para simular como diferentes modelos de IA performam melhor em diferentes combinações de hardware.



Peças configuráveis

CPU, RAM, GPU, VRAM e demais componentes.



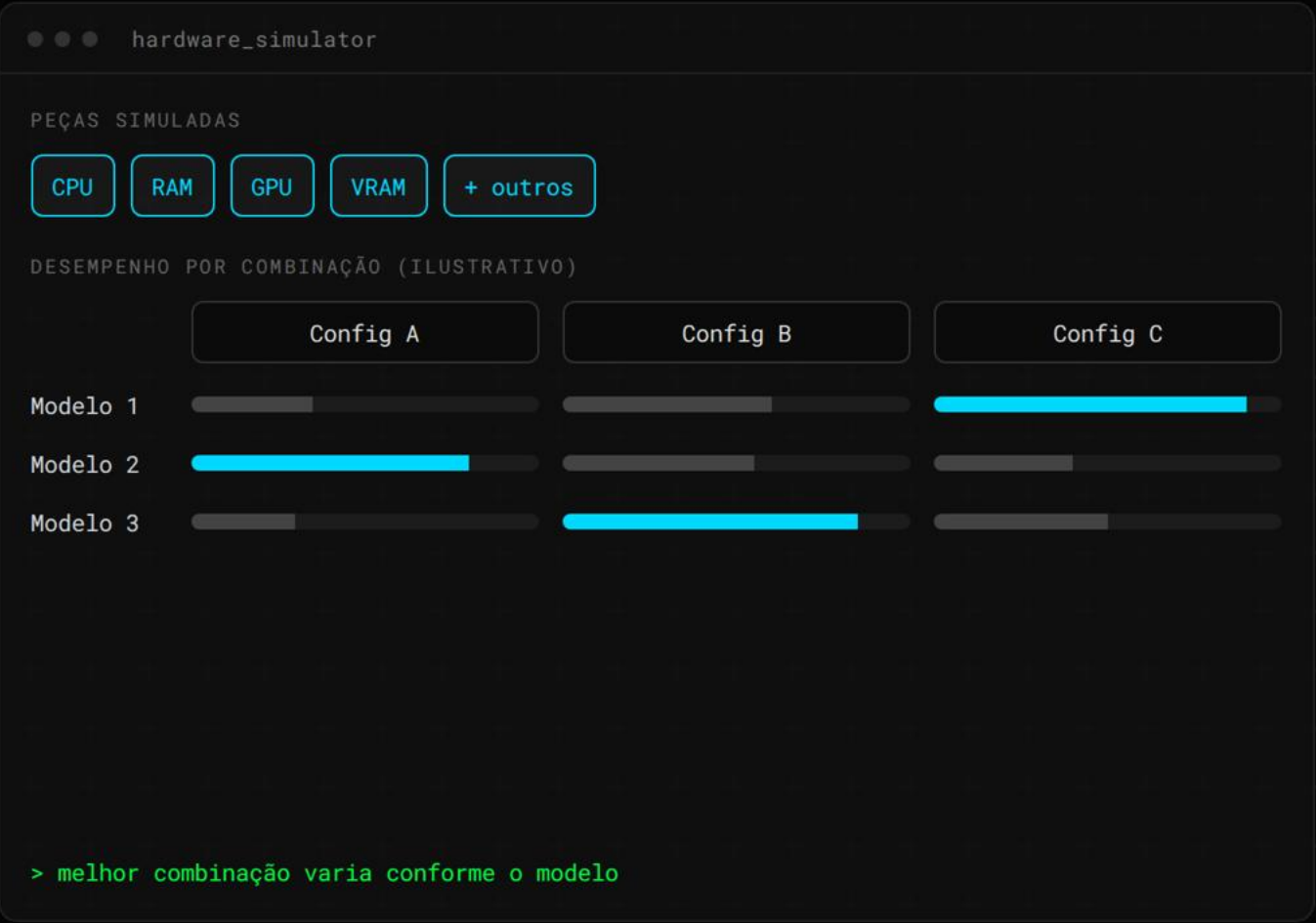
Combinações de hardware

Compara cenários montados a partir das peças.



Modelo x hardware

Mostra qual combinação favorece cada modelo de IA.



Atualização de Documentação

Artefato que define e explicita os princípios, responsabilidades, controles e procedimentos utilizados pelo MLX Pilot para garantir o uso seguro, transparente, privado e auditável de modelos de inteligência artificial.

- **Escopo de IA**
Delimita o uso de modelos de IA no MLX Pilot.
- **LGPD**
Diretrizes de privacidade e proteção de dados pessoais.
- **Classificações de risco**
Categorização dos riscos do uso dos modelos.

/MLX-Pilot-Docs

- /MLX-Pilot-Docs
 - 01 Princípios
 - 02 Responsabilidades
 - 03 Controles
 - 04 Procedimentos
 - 05 Escopo de IA
 - 06 LGPD
 - 07 Classificação de risco

GARANTIAS

- Seguro
- Transparente
- Privado
- Auditável

Memória Local, Sessão Ativa e Contexto

PR #15 · PR #31

Agora, as sessões são persistidas localmente para permitir a continuação de conversas e interações com modelos de IA sem perder contexto.



Persistência local

Sessões gravadas na máquina do usuário.



Sessão ativa

Retoma a conversa exatamente de onde parou.



Contexto preservado

Interações com os modelos seguem sem perder contexto.

session_store

SALVAS NO DISCO

Sessão #1

Sessão #2 · ATIVA

Sessão #3

LOG

```
> app reiniciado
> session.load("#2") OK
> contexto restaurado OK
> conversa retomada
```

0 bytes enviados para fora da máquina

RASTREABILIDADE

PR #15

PR #31

SYSTEM STATUS: DELIVERED

RUST TAURI METAL

Integração com n8n

ISSUE #36 · PR #41

O MLX-Pilot possui integração direta com o n8n local, podendo gerar workflows via prompt e um editor visual nativo para criar um workflow do zero.



n8n local

Integração direta com a instância local do n8n.



Workflow via prompt

Descreva a automação e o workflow é gerado.



Editor visual nativo

Crie um workflow do zero dentro do app.

n8n_integration

1 · GERAR VIA PROMPT

Prompt: "Resumir documentos novos e enviar por e-mail"

Trigger



LLM local



E-mail

workflow gerado

2 · EDITOR VISUAL NATIVO

Trigger



+ adicionar nó

n8n local · integração direta

RASTREABILIDADE

Issue #36

PR #41

SYSTEM STATUS: DELIVERED

RUST

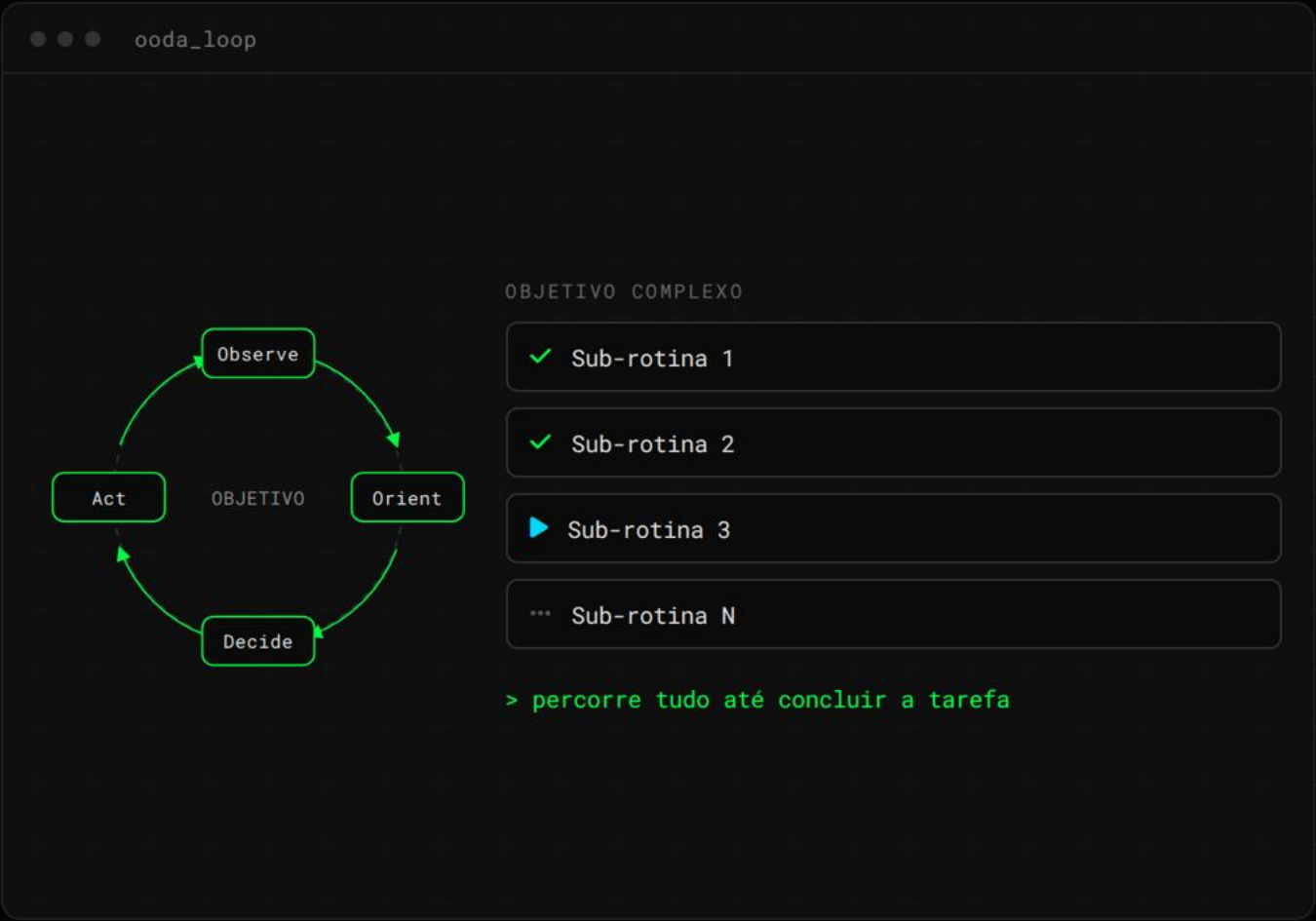
TAURI

APPLE METAL

Loop OODA

Implementação de ciclo OODA (Observe-Orient-Decide-Act) para tarefas multi-etapa. Dentro de um loop OODA, a IA pode desmembrar um objetivo complexo em várias sub-rotinas e percorrer todas elas de forma automatizada até terminar a tarefa.

- **Tarefas multi-etapa**
Objetivo complexo dividido em sub-rotinas.
- **Ciclo automatizado**
Observe, Orient, Decide e Act a cada sub-rotina.
- **Até concluir**
Percorre todas as sub-rotinas até o fim da tarefa.



Objetivo Acadêmico

RESEARCH_FOCUS // EXPERIMENTATION // BENCHMARK



01

Arquitetura Modular

Estudo aprofundado sobre desacoplamento de sistemas, separando interface (Tauri) do controle lógico (Rust) e inferência.



02

Local vs Remoto

Análise comparativa crítica entre execução local (privacidade, latência zero) e nuvem (custo, escalabilidade).



03

Benchmark

Coleta sistemática de métricas de performance: Tokens por segundo (TPS), Tempo para o primeiro token (TTFT) e uso de memória.



04

Stress Test GPU

Avaliação de limites de hardware com modelos massivos (7B+), monitorando temperatura, consumo energético e gargalos de VRAM.



05

Base de Pesquisa

Criação de uma plataforma estável e documentada para servir como fundação para futuros Trabalhos de Conclusão de Curso (TCC).

Alinhamento com NVIDIA

CUDA_ACCELERATION_ENABLED = TRUE

RTX 4090 A100 TENSOR



Modelos Large Scale

Viabilizar testes de inferência com modelos acima de 7B parâmetros (Llama-3-70B, Mixtral 8x7B) utilizando aceleração de hardware dedicada.



Análise de VRAM

Monitoramento granular do consumo de memória de vídeo durante carregamento de camadas (offloading) e contexto estendido.



CUDA vs CPU

Benchmark comparativo de throughput (tokens/s) entre execução puramente em CPU e pipelines acelerados via CUDA.



Validação Acadêmica

Geração de dados empíricos para artigos sobre eficiência energética e latência em inferência local de LLMs.

🕒Inference Speed (Tokens/s)

Model: Llama-3-8B-Instruct-Q4

Backlog Atual

SPRINT: ATUAL // PI_IV

● High Priority ● Medium Priority

BACKLOG / FUTURO5	SPRINT ATUAL (ALTA PRIORIDADE)5	CONCLUÍDO20
AGENT[S4] L Ferramentas sandboxed para o agente SECURITY[S4] L Políticas, segurança e secrets vault CORE[S4] L Estruturar agent core local LOGIC[S4] M Sistema de skills e carregamento de	BENCHMARK#IN_PROGRESS Benchmark comparativo de throughput e latencia Métricas S3 TESTING#IN_PROGRESS Stress test de VRAM e estabilidade termica Métricas S3 UI#IN_REVIEW Console de observabilidade do agente	[S2] M✓ DONE OpenClaw com logs em tempo real e chat nativo [S2] M✓ DONE Diagnosticos, onboarding e install state [S2] M✓ DONE Gerenciamento de modelos instalados [S2] M✓ DONE

Expansão Acadêmica

PLANNING_PHASE



PRIORIDADE MÉDIA

Benchmark Comparativo

Execução de baterias de testes padronizados para medir TPS (Tokens Per Second) e latência entre diferentes backends (MLX vs Llama.cpp) e hardwares.

METRICS_COLLECTION →



PRIORIDADE MÉDIA

Stress Test GPU

Simulação de carga máxima em VRAM com modelos de parâmetros elevados (>13B) para validar estabilidade térmica e gerenciamento de memória.

HARDWARE_LIMITS →



PRIORIDADE MÉDIA

Observabilidade

Implementação de sistema robusto de logs estruturados e métricas em tempo real para monitorar o comportamento do daemon e inferências.

TELEMETRY_SYSTEM →



PRIORIDADE MÉDIA

Hardening de Segurança

Reforço das camadas de segurança, incluindo sanitização rigorosa de inputs no chat e isolamento (sandbox) do ambiente de execução de código.

SECURITY_AUDIT →

Evolução do Projeto

ROADMAP_TARGETS // v1.0.0-release

IN DEVELOPMENT



PHASE 1

Agente Autônomo

Loop OODA já implementado (PR #35). Próximo passo: ferramentas sandboxed e sistema de skills do agente.



PHASE 3

Sistema de Plugins

Arquitetura extensível baseada em WebAssembly (WASM) para permitir que a comunidade crie novos providers e ferramentas sem recompilar o core.



PHASE 2

n8n Embedded

n8n local já integrado (PR #41). Próximo passo: embutir o motor no binário.



PHASE 4

Multiplataforma

Builds otimizadas para Windows (CUDA/DirectML) e Linux, com instaladores nativos (.msi, .deb) e gerenciamento de dependências isolado.

Impacto Acadêmico

EDUCATION // RESEARCH // INNOVATION



Laboratório Prático de IA

REQ: GPU/CPU

Ambiente controlado para alunos experimentarem com LLMs sem custos de nuvem. Permite "botar a mão na massa" com inferência, prompts e fine-tuning básico.



Base para TCCs

DOCS: READY

Framework extensível que serve de ponto de partida para monografias. Alunos podem focar em desenvolver novos plugins ou agentes ao invés de configurar infraestrutura.



Uso em Disciplinas

COURSE: AI_101

Ferramenta didática para aulas de Inteligência Artificial, Sistemas Distribuídos e Compiladores, demonstrando conceitos teóricos na prática.

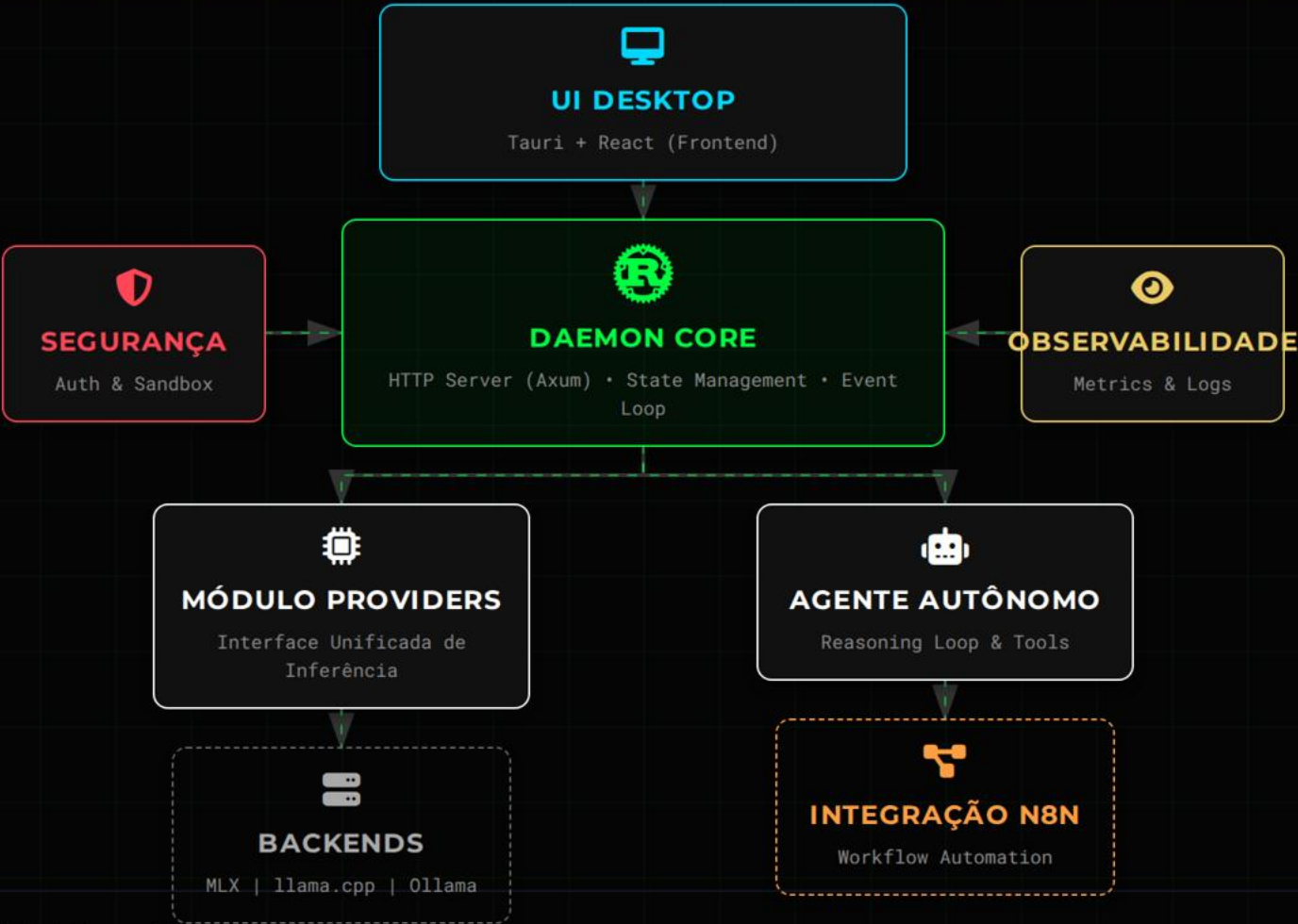


Plataforma Institucional

SCOPE: CAMPUS

Hub centralizado para pesquisa e desenvolvimento dentro da universidade, fomentando grupos de estudo e parcerias com empresas (ex: NVIDIA).

Arquitetura Futura



Tecnologias Utilizadas

HIGH_PERFORMANCE // MEMORY_SAFE // LOCAL_FIRST

CORE FOUNDATION



v1.75+

Rust

Linguagem principal garantindo segurança de memória, performance nativa e zero-cost abstractions.

Core Logic

Daemon



v2.0-beta

Tauri

Framework para construção de UI desktop leve e segura utilizando tecnologias web (HTML/JS/CSS).

Frontend

WebView



v0.7

Axum

Servidor HTTP ergonômico e modular para comunicação robusta entre UI e Daemon via API REST/SSE.

API REST

Handlers



Runtime

Tokio

Runtime assíncrono para gerenciar concorrência, I/O não bloqueante e tarefas em background.

Async

Concurrency

INFERENCE ENGINES



MLX

APPLE RESEARCH

Framework de array otimizado para Apple Silicon, permitindo inferência de alta performance em Macs (M1/M2/M3).

Metal API



llama.cpp

GGML / GGUF

Motor de inferência C++ puro com suporte a quantização (GGUF), rodando eficientemente em CPU e GPU variadas.

Cross-platform



Ollama

MODEL RUNNER

Interface de alto nível para gerenciamento simplificado de modelos locais e compatibilidade de API.

Easy Setup

Métricas e Performance

HARDWARE: APPLE M3 MAX (128GB RAM) // CUDA: N/A // METAL: ENABLED



BEST: MLX

62.4 tok/s

PEAK THROUGHPUT

Model: Llama-3-8B-Instruct (4-bit)



LOWEST: LLAMA.CPP

12ms

P50 LATENCY

18ms

P95 LATENCY



EFFICIENCY

4.8 GB

AVG VRAM USAGE

Conclusão do Projeto

STATUS: READY_FOR_DEPLOYMENT // PI_IV_IN_PROGRESS

FUNCTIONAL SOFTWARE



Plataforma Técnica Real

O MLX-Pilot não é apenas um conceito teórico. É um software funcional capaz de baixar, carregar e executar inferência de LLMs localmente com performance mensurável.

MODULAR ARCHITECTURE



Projeto Escalável

A arquitetura baseada em *Providers* e desacoplamento entre UI (Tauri) e Core (Rust) permite a fácil adição de novos backends e funcionalidades sem refatoração massiva.

PRODUCTION READY CORE



Arquitetura Robusta

O uso de Rust garante segurança de memória (memory safety) e alta performance, criando uma fundação sólida para transformar o protótipo acadêmico em um produto.

ACADEMIC VALUE



Laboratório de IA

Potencial para se tornar a ferramenta padrão da instituição para ensino de IA Generativa, permitindo que alunos experimentem sem custos de nuvem ou configurações complexas.

Próximos Passos

SPRINT_STATUS: ACTIVE

Consolidação

O Projeto Integrador IV foca na estabilização da arquitetura, na evolução do agente e na coleta de métricas para validação acadêmica. O objetivo é entregar um artefato robusto e bem documentado.

PROJECT COMPLETION

85%

```
$ ./mlx-pilot --benchmark
Loading model weights... OK
Initializing Metal backend... OK
Running inference test [batch=1]...
Processing tokens...
```



Finalização do Benchmark

IN PROGRESS

Coleta sistemática de métricas de latência e consumo de VRAM em diferentes hardwares.



Implementação de Observabilidade

PENDING

Integração de dashboards para visualização de logs estruturados e tracing de requests.



Evolução do Agente

PLANNED

Ferramentas sandboxed, sistema de skills e API do agente integrada ao daemon, sobre o loop OODA já implementado.



Empacotamento e Entrega Final

PLANNED

Empacotamento, documentação técnica final e planejamento de entrega.